NEXT ENGINEER · _SHEERAT_

# Smart Manufacturing for Industry 4.0

**BOE080**

One-day revision notes: definitions, solved examples, code and important questions

**Program:** B.Tech | **Branches:** ALL | **Semester:** 7 | **Credits:** 3 | **L-T-P:** 3-0-0

**Scheme:** AKTU B.Tech 4th year, Open Elective-II, 2025-26 onwards

| UNIT 1 | UNIT 2 | UNIT 3 | UNIT 4 | UNIT 5 |
|---|---|---|---|---|
| Introduction to Industry 4.0 | Automation, PLCs and Cyber-Physical Systems | Industrial Communication Protocols | IoT Platforms and Gateways | Machine Learning for Smart Manufacturing |

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: architectures, protocols and comparison tables.
3. **Solved examples**: the calculations and small programs that come up (OEE, takt time, ladder logic, sensor scaling, MQTT topics and QoS, OPC UA address space, linear regression, decision tree split).
4. **Code / key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

## How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (Industry 4.0, globalization, four industrial revolutions, lean, smart and connected business, smart factory) and Unit 2 (PLCs, ladder logic, sensors, HMI, CPS, ISA-95, 5C architecture, digital twin).
- **Afternoon (3 hours):** Unit 3 (MQTT, OPC UA, EtherNet/IP, PROFINET, EtherCAT, AR) and Unit 4 (IoT platforms, data modelling, thing abstraction, gateways, Mosquitto, programming).
- **Evening (2 hours):** Unit 5 (machine learning types and models: ANN, decision trees, regression, genetic algorithms). Then the "Last-hour checklist".
- **In the exam:** draw the ISA-95 pyramid, the 5C pyramid, the MQTT publish-subscribe diagram and a ladder rung; they are quick marks.

**AKTU paper pattern (theory, 70 marks, 3 hours):** Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

# UNIT 1 Introduction to Industry 4.0

## ✍️ 2-mark definitions

- **Industry 4.0:** the fourth industrial revolution, in which manufacturing is transformed by connecting machines, products and people through cyber-physical systems, IoT, cloud, big data and AI, enabling self-optimizing, flexible production. The term came from the German government's "Industrie 4.0" strategy (2011).
- **Smart factory:** a highly digitized, connected production facility where machines, systems and products communicate and make decentralized decisions in real time.
- **Industrial IoT (IIoT):** the use of IoT technology (sensors, connectivity, analytics) in industrial settings.
- **Lean manufacturing:** a production philosophy (from the Toyota Production System) that maximizes customer value by eliminating waste (muda).
- **Globalization:** the integration of economies through trade, investment, technology and supply chains across countries.
- **Mass customization:** producing customized products at near mass-production cost, made possible by flexible automation.
- **Interoperability:** the ability of machines, devices and software from different vendors to exchange and use data.
- **Vertical integration:** connecting all levels of a company, from shop-floor sensors up to ERP.
- **Horizontal integration:** connecting the whole value chain across companies: suppliers, manufacturers, logistics and customers.

## 💡 Core concepts

### Four industrial revolutions

| Revolution | Period | Key technology |
|---|---|---|
| Industry 1.0 | Late 18th century | Mechanization with water and steam power |
| Industry 2.0 | Late 19th century | Electricity, assembly lines, mass production |
| Industry 3.0 | 1970s | Electronics, IT, PLCs, automation |
| Industry 4.0 | 2010s | Cyber-physical systems, IoT, cloud, AI, big data |
| Industry 5.0 (emerging) | 2020s | Human-centric, sustainable, resilient; humans and cobots together |

**Nine technology pillars of Industry 4.0 (BCG):** autonomous robots, simulation, horizontal and vertical system integration, industrial IoT, cybersecurity, cloud computing, additive manufacturing (3D printing), augmented reality, big data and analytics.

**Design principles:** interoperability, virtualization (digital twins), decentralization (local decisions), real-time capability, service orientation, modularity.

**Globalization and emerging issues:** global supply chains and competition push for lower cost, faster delivery and customization; issues: supply chain disruptions (pandemic, geopolitics), cybersecurity, skill gaps, job displacement, data ownership, standards, energy and sustainability, SMEs' difficulty adopting costly technology, reshoring and "Make in India" / PLI schemes.

**Lean manufacturing**

- Principles (Womack and Jones): identify value, map the value stream, create flow, establish pull, seek perfection.
- Seven wastes (TIMWOOD): Transport, Inventory, Motion, Waiting, Over-production, Over-processing, Defects (plus unused talent).
- Tools: 5S (Sort, Set in order, Shine, Standardize, Sustain), kanban, JIT, kaizen, poka-yoke (mistake proofing), SMED (quick changeover), value stream mapping, Andon, TPM.
- **Lean 4.0:** digital tools strengthen lean: e-kanban, real-time OEE dashboards, sensor-based poka-yoke, digital value stream maps.

**Smart and connected business perspective:** products become connected services (servitization: selling "power by the hour" instead of engines), new business models (pay-per-use, predictive maintenance contracts), data-driven decisions, customer integration, platform ecosystems.

**Smart factory characteristics:** connected (sensors everywhere), optimized (self-adjusting processes), transparent (real-time visibility through dashboards), proactive (predictive maintenance and quality), agile (quick reconfiguration). Components: CPS, IIoT, MES, robotics and cobots, AGVs/AMRs, additive manufacturing, cloud and edge computing, digital twins, AR for workers.

## 🧮 Solved examples

**Example 1: OEE (Overall Equipment Effectiveness).** Planned time 480 min; downtime 80 min; ideal cycle time 1 min/unit; units produced 360; good units 342.

- Availability = (480 − 80)/480 = 400/480 = 0.833.
- Performance = (1 × 360)/400 = 0.90.
- Quality = 342/360 = 0.95.
- **OEE = 0.833 × 0.90 × 0.95 = 0.713 (71.3%)**. World-class OEE is about 85%.

**Example 2: Takt time.** Available time 450 min per shift, customer demand 225 units per shift. Takt time = 450 / 225 = **2 min per unit**: the line must finish one unit every 2 minutes.

## Key points to remember

- Industry 1.0 steam → 2.0 electricity → 3.0 electronics/IT → 4.0 CPS/IoT/AI.
- Nine pillars; six design principles.
- Lean: five principles, TIMWOOD wastes, 5S, kanban, poka-yoke, SMED.
- OEE = Availability × Performance × Quality.
- Vertical vs horizontal integration.

## ⭐ Important questions

1. What is Industry 4.0? Explain the four industrial revolutions and the nine pillars of Industry 4.0.
2. Explain globalization and emerging issues in manufacturing.
3. Explain lean manufacturing and its relation with Industry 4.0.
4. Explain smart and connected business perspectives.
5. What is a smart factory? Explain its characteristics and components.

NEXT ENGINEER
_sheerat_

# UNIT 2 Automation, PLCs and Cyber-Physical Systems

## ✍️ 2-mark definitions

- **Automation:** using control systems and technology to operate machines and processes with minimal human intervention.
- **PLC (Programmable Logic Controller):** a rugged industrial computer that reads inputs, executes a user program cyclically and drives outputs to control machines and processes.
- **Scan cycle:** the repeated PLC cycle: read inputs → execute program → update outputs → housekeeping/communication.
- **Ladder logic (LD):** a graphical PLC language resembling relay circuits, with contacts (inputs) and coils (outputs) on rungs.
- **IEC 61131-3:** the standard for PLC programming languages: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List (deprecated), Sequential Function Chart.
- **Smart sensor:** a sensor with built-in processing and communication that can self-calibrate, diagnose and send digital data (often via IO-Link).
- **HMI (Human-Machine Interface):** the screen or panel through which operators monitor and control a machine or process.
- **SCADA:** Supervisory Control and Data Acquisition, software that collects data from PLCs and RTUs across a plant and supervises the process.
- **Cyber-Physical System (CPS):** a system in which computation and networking are tightly integrated with physical processes through sensors and actuators, with feedback loops.
- **ISA-95 (IEC 62264):** the international standard for integrating enterprise and control systems, defining levels 0–4.
- **Digital twin:** a virtual model of a physical asset, process or system that is continuously updated with real data and used for monitoring, simulation and prediction.

## 💡 Core concepts

**PLC architecture:** power supply, CPU (processor and memory), input modules (digital: switches, sensors; analog: 4–20 mA, 0–10 V), output modules (relays, transistors, analog), communication modules (Ethernet, PROFINET, Modbus), programming device. Types: compact (fixed I/O), modular (rack), safety PLCs. Vendors: Siemens (S7, TIA Portal), Allen-Bradley/Rockwell (Studio 5000), Mitsubishi, Schneider, Delta. Open-source/free tools: OpenPLC, CODESYS (free development system).

**Ladder logic elements:** normally open contact (–| |–), normally closed contact (–|/|–), output coil (–( )–), set/reset (latch/unlatch) coils, timers (TON on-delay, TOF off-delay, TP pulse), counters (CTU up, CTD down), comparison and math blocks.

**Communication between devices and PLCs:** hard-wired I/O (24 V DC signals, 4–20 mA), fieldbuses (PROFIBUS, Modbus RTU, DeviceNet), industrial Ethernet (PROFINET, EtherNet/IP, Modbus TCP, EtherCAT), IO-Link for smart sensors, OPC UA and MQTT to higher systems.

**Sensors and smart sensors:** proximity (inductive, capacitive), photoelectric, limit switches, encoders, temperature (RTD, thermocouple), pressure, flow, level, vibration (accelerometers), vision cameras.

Smart sensors add identification, diagnostics, parameter setting and digital output; IO-Link allows remote configuration and condition data.

**HMI design principles:** clear hierarchy (overview → area → detail), consistent colours (grey normal, colours only for abnormal states and alarms: high-performance HMI, ISA-101), alarm management (prioritization, avoid floods), trends instead of only numbers, large touch targets, navigation consistency, role-based access.

**CPS key components:** physical layer (machines, sensors, actuators), network layer (communication), cyber layer (computation, data storage, analytics, models), feedback control and human interface.

**ISA-95 levels (automation pyramid)**

| Level | Function | Systems | Time frame |
|---|---|---|---|
| 4 | Business planning and logistics | ERP | Months, weeks, days |
| 3 | Manufacturing operations management | MES, MOM | Days, shifts, hours |
| 2 | Monitoring and supervisory control | SCADA, HMI | Hours, minutes, seconds |
| 1 | Sensing and manipulating | PLC, DCS controllers | Seconds, milliseconds |
| 0 | Physical production process | Sensors, actuators, machines | Real time |

In Industry 4.0 the strict pyramid becomes a flatter network where data flows directly (via OPC UA or MQTT) to cloud and analytics.

**CPS 5C architecture (Lee, Bagheri, Kao, 2015)**

| Level | Name | Function |
|---|---|---|
| 1 | Smart Connection | Acquire accurate, reliable data from machines and sensors |
| 2 | Data-to-Information Conversion | Extract meaningful information: health values, remaining useful life |
| 3 | Cyber | Central hub: compare machines, build twin models, fleet analytics |
| 4 | Cognition | Present knowledge to experts for decisions (visualization, diagnostics, prioritization) |
| 5 | Configuration | Feed decisions back to the physical space: self-configure, self-adjust, self-optimize |

**Digital twin:** types: component/asset twin, product twin, process/production twin, system twin. Uses: virtual commissioning, predictive maintenance, what-if simulation, design optimization, operator training. Difference from simulation: a twin is continuously connected to and synchronized with the real asset.

## 🧮 Solved examples

**Example 1: Start-stop motor circuit (ladder logic).** Start push button (NO, I0.0), stop push button (NC wired, used as a NO contact I0.1 that is closed when not pressed), motor contactor Q0.0. The motor latches on when Start is pressed and drops out when Stop is pressed.

```
Rung 1:
   I0.0 Start      I0.1 Stop        Q0.0 Motor
 ---| |-------+------| |----------------( )---
              |
   Q0.0       |
 ---| |-------+
```

Logic: Q0.0 = (I0.0 OR Q0.0) AND I0.1. The Q0.0 contact in parallel with Start is the seal-in (holding) contact.

**Example 2: Timer.** A conveyor must run for 10 s after a part sensor (I0.2) turns off. Use an off-delay timer TOF with preset 10 s: input I0.2, output drives Q0.1. The same logic in Structured Text:

```
T1(IN := PartSensor, PT := T#10s);
Conveyor := T1.Q;
```

**Example 3: Analog scaling.** A pressure transmitter (0–10 bar) gives 4–20 mA; the PLC reads 4 mA as 0 counts and 20 mA as 27,648 counts (Siemens). A reading of 16,589 counts = 16,589 / 27,648 = 0.6 of span = **6 bar** (current 4 + 0.6 × 16 = 13.6 mA).

## Key points to remember

- PLC scan: input → program → output.
- IEC 61131-3 languages: LD, FBD, ST, SFC (IL deprecated).
- Ladder start-stop with seal-in contact.
- Smart sensors and IO-Link; HMI follows ISA-101 (grey screens, alarm colours).
- ISA-95 levels 0–4: field, control, SCADA, MES, ERP.
- 5C: Connection, Conversion, Cyber, Cognition, Configuration.
- Digital twin = continuously synchronized virtual model.

## ⭐ Important questions

1. Explain automation with PLCs: architecture, scan cycle and programming languages.
2. Write ladder logic for given control problems (start-stop, timers, counters).
3. Explain communication between devices and PLCs. Explain sensors and smart sensors.
4. Explain HMI design principles.
5. Explain cyber-physical systems, their components, the ISA-95 architecture and the 5C architecture.
6. What is a digital twin? Explain its types and applications.

NEXT ENGINEER _sheerat_

# UNIT 3 Industrial Communication Protocols

## ✍️ 2-mark definitions

- **MQTT (Message Queuing Telemetry Transport):** a lightweight publish-subscribe messaging protocol over TCP, designed for constrained devices and unreliable networks, with a broker that routes messages by topic.
- **Broker:** the MQTT server that receives published messages and forwards them to subscribers of matching topics (Mosquitto, HiveMQ, EMQX).
- **Topic:** a hierarchical string such as `plant1/line2/press/temp` used to route MQTT messages; wildcards `+` (single level) and `#` (multi-level).
- **QoS (Quality of Service):** the delivery guarantee level in MQTT: 0 at most once, 1 at least once, 2 exactly once.
- **Retained message:** the last message on a topic stored by the broker and sent immediately to new subscribers.
- **Last Will and Testament (LWT):** a message the broker publishes on behalf of a client that disconnects unexpectedly.
- **OPC UA (Open Platform Communications Unified Architecture):** a platform-independent, secure, service-oriented standard (IEC 62541) for industrial data exchange with a rich information model.
- **EtherNet/IP:** industrial Ethernet protocol using the Common Industrial Protocol (CIP) over standard Ethernet, TCP and UDP (ODVA, Rockwell).
- **PROFINET:** industrial Ethernet standard from PROFIBUS and PROFINET International (Siemens), with real-time (RT) and isochronous real-time (IRT) classes.
- **EtherCAT:** Ethernet for Control Automation Technology (Beckhoff), very fast "processing on the fly" protocol where one frame passes through all slaves.
- **Augmented reality (AR):** overlaying digital information (text, 3D models) on the real-world view through phones, tablets or smart glasses.

## 💡 Core concepts

**MQTT history:** created in 1999 by Andy Stanford-Clark (IBM) and Arlen Nipper for monitoring oil pipelines over satellite links; became an OASIS standard (3.1.1 in 2014, 5.0 in 2019); widely used in IoT and Industry 4.0.

**MQTT message types (control packets):** CONNECT, CONNACK, PUBLISH, PUBACK, PUBREC, PUBREL, PUBCOMP, SUBSCRIBE, SUBACK, UNSUBSCRIBE, UNSUBACK, PINGREQ, PINGRESP, DISCONNECT (and AUTH in v5).

**MQTT QoS levels**

| QoS | Guarantee | Handshake | Use |
|---|---|---|---|
| 0 | At most once (fire and forget) | PUBLISH only | Frequent sensor readings where loss is acceptable |

| QoS | Guarantee | Handshake | Use |
|---|---|---|---|
| 1 | At least once (duplicates possible) | PUBLISH → PUBACK | Most telemetry and commands |
| 2 | Exactly once | PUBLISH → PUBREC → PUBREL → PUBCOMP | Billing, critical commands |

**MQTT features:** small 2-byte fixed header, keep-alive, persistent sessions, retained messages, LWT, TLS security and username/password or certificates. Applications: remote monitoring, energy metering, fleet tracking, smart factory data to cloud, Sparkplug B for industrial payloads.

**OPC UA**

- History: classic OPC (1996, based on Microsoft COM/DCOM, Windows only: OPC DA, HDA, A&E); OPC UA released in 2008 as platform-independent and secure.
- Specification parts: concepts, security model, address space model, services, information model, mappings (binary TCP, HTTPS, JSON), profiles, data access, alarms and conditions, programs, historical access, pub-sub (Part 14).
- **Address space:** nodes (objects, variables, methods, object types, variable types, reference types, data types, views) connected by references; each node has a NodeId and attributes.
- **Client-server:** clients browse, read, write, subscribe (monitored items) and call methods on servers embedded in PLCs, gateways or software.
- **Security:** authentication (certificates, user tokens), signing and encryption of messages, security policies.
- **Companion specifications:** standard information models for machine types (robotics, machine tools: umati, packaging: PackML).
- Programming with free/open-source tools: open62541 (C), python-opcua/asyncua, Eclipse Milo (Java), Node-OPCUA; proprietary: Siemens, Kepware KEPServerEX, Unified Automation SDKs, UaExpert client.

**Industrial Ethernet comparison**

| Protocol | Organization | Real-time | Notes |
|---|---|---|---|
| EtherNet/IP | ODVA | Soft real-time (ms) | CIP over TCP/UDP, standard switches |
| PROFINET | PI (Siemens) | RT about 1–10 ms, IRT under 1 ms | Integrates with PROFIBUS, TIA Portal |
| EtherCAT | ETG (Beckhoff) | Hard real-time, cycle under 100 µs | On-the-fly processing, motion control |
| Modbus TCP | Modbus.org | Soft | Simple register-based, widely supported |
| OPC UA (TSN) | OPC Foundation | With TSN, deterministic | Vertical integration, semantic models |
| MQTT | OASIS | Not real-time | Telemetry to cloud |

**Augmented reality in manufacturing:** step-by-step assembly instructions overlaid on the product, remote expert assistance, maintenance guides showing live sensor values on the machine, training,

quality inspection (comparing CAD overlay to part), warehouse picking. Devices: tablets, HoloLens, RealWear; platforms: Vuforia, PTC.

## 🧮 Solved examples

**Example 1: MQTT topic design.** Plant 1 has two lines, each with a press and an oven. Topics: `plant1/line1/press/temp`, `plant1/line1/oven/temp`, `plant1/line2/press/temp`, `plant1/line2/press/status`.

- Subscribe `plant1/+/press/temp` → both presses' temperatures.
- Subscribe `plant1/line2/#` → everything on line 2.
- Subscribe `plant1/+/+/temp` → all temperatures.

**Example 2: QoS message count.** Sending 1 message at QoS 2 needs 4 packets between publisher and broker (PUBLISH, PUBREC, PUBREL, PUBCOMP); QoS 1 needs 2; QoS 0 needs 1. A sensor sending 10 readings per second at QoS 2 would create 40 packets per second on that link, which is why QoS 0 or 1 is used for high-rate telemetry.

**Example 3: OPC UA address space for a motor.**

```
Objects
 └── Line1
      └── Motor1 (Object, type MotorType)
           ├── Speed      (Variable, Double, rpm, NodeId ns=2;s=Line1.Motor1.Speed)
           ├── Current    (Variable, Double, A)
           ├── Running    (Variable, Boolean)
           └── Start()    (Method)
```

A client subscribes to Speed and Current with a 500 ms sampling interval and calls Start() to start the motor.

## Code

**Publishing machine data with MQTT (Python, paho-mqtt)**

```python
import json, time, random
import paho.mqtt.client as mqtt

client = mqtt.Client(client_id="press-01")
client.will_set("plant1/line1/press/status", "offline", qos=1, retain=True)
client.connect("localhost", 1883)
client.publish("plant1/line1/press/status", "online", qos=1, retain=True)

while True:
    reading = {"temp": round(60 + random.random() * 5, 2), "ts": time.time()}
    client.publish("plant1/line1/press/temp", json.dumps(reading), qos=1)
    time.sleep(1)
```

**Reading a value from an OPC UA server (Python, asyncua)**

```
import asyncio
from asyncua import Client

async def main():
    async with Client("opc.tcp://192.168.0.10:4840") as c:
        node = c.get_node("ns=2;s=Line1.Motor1.Speed")
        print("Speed:", await node.read_value())

asyncio.run(main())
```

## Key points to remember

- MQTT: pub-sub, broker, topics with + and #, QoS 0/1/2, retained, LWT, keep-alive; 1999 IBM, OASIS standard.
- OPC UA: platform-independent, secure, information model, address space of nodes, client-server and pub-sub.
- EtherNet/IP (CIP), PROFINET (RT/IRT), EtherCAT (on-the-fly, fastest), Modbus TCP.
- AR: instructions, remote assistance, maintenance, training.

## ⭐ Important questions

1. Explain MQTT: history, broker, message types, QoS levels and applications.
2. Explain OPC UA: history, specification, information model, client and server.
3. Compare industrial communication protocols: EtherNet/IP, PROFINET, EtherCAT, Modbus TCP, OPC UA, MQTT.
4. Explain how OPC UA and MQTT are programmed using open-source and proprietary software.
5. Explain the use of augmented reality in smart manufacturing.

# UNIT 4 IoT Platforms and Gateways

## ✍️ 2-mark definitions

- **IoT platform:** software (often cloud-based) that connects devices, collects and stores their data, manages them and provides analytics, dashboards and APIs (AWS IoT Core, Azure IoT Hub, Siemens Insights Hub (MindSphere), PTC ThingWorx, ThingsBoard).
- **Data modelling:** defining how device data is structured: entities, attributes, telemetry, units, relationships and metadata.
- **Thing abstraction:** representing a physical device as a software object (a "thing" or "device twin/shadow") with properties, events and actions, independent of its hardware.
- **Device shadow (twin):** a cloud-side JSON document holding a device's last reported and desired state.
- **IoT gateway:** a device that connects field devices and machines (using Modbus, OPC UA, BLE) to the cloud, translating protocols, buffering and processing data at the edge.
- **Edge computing:** processing data close to the machines to reduce latency and bandwidth.
- **Machine interface:** the connection point through which data is read from a machine (PLC port, OPC UA server, fieldbus, analog sensors, MTConnect).
- **Mosquitto:** an open-source MQTT broker from the Eclipse Foundation, lightweight and widely used, deployable locally or on a cloud VM.

## 💡 Core concepts

**Basic functionalities of an IoT platform:** device connectivity and management (registration, authentication, firmware updates), data ingestion (MQTT, HTTP, CoAP), storage (time-series databases), rules engine (if temperature > 80 then alert), analytics and ML, visualization (dashboards), APIs and integration with ERP/MES, security.

**Data modelling for IIoT**

- Telemetry (time series: temperature, vibration), attributes (static: model, serial number, location), events (alarms, state changes), commands (start, setpoint).
- Standards: OPC UA information models, Asset Administration Shell (AAS, the Industry 4.0 digital twin standard from Plattform Industrie 4.0), Sparkplug B for MQTT, W3C Web of Things Thing Description.
- Good practice: consistent units and naming, timestamps in UTC, quality flags, hierarchical asset model (enterprise → site → area → line → machine).

**Thing abstraction:** a "thing" has properties (readable/writable state), events (notifications) and actions (callable functions). Apps interact with the abstraction, not the device protocol, so devices can be replaced without changing apps. Example: W3C WoT Thing Description in JSON-LD.

**Networks in IIoT:** wired (Ethernet, fieldbuses), wireless (Wi-Fi, Bluetooth LE, Zigbee, WirelessHART, ISA100.11a, LoRaWAN, NB-IoT, private 5G), TSN for deterministic Ethernet.

**IoT gateways and machine interfaces**

- Functions: protocol conversion (Modbus/OPC UA/PROFINET to MQTT/HTTP), data filtering and aggregation, local storage and buffering during network outages, edge analytics, security (firewall, VPN, TLS), device management.
- Examples: Siemens IoT2050, Moxa, Advantech, Raspberry Pi with Node-RED, AWS Greengrass, Azure IoT Edge.
- Brownfield connectivity: older machines without networking are retrofitted with external sensors (current clamps, vibration sensors) or by reading the PLC.

**Cloud-based Mosquitto brokers:** install Mosquitto on a cloud VM (AWS EC2, Azure VM) or use a managed service; configure listeners (1883 plain, 8883 TLS), authentication (password file, certificates), access control lists per topic, persistence, bridging (local broker bridged to cloud broker). Test with `mosquitto_pub` and `mosquitto_sub`.

**Programming with free and proprietary software:** Node-RED (flow-based programming, free), Python (paho-mqtt, asyncua, pandas), InfluxDB + Grafana (time series and dashboards), ThingsBoard (open-source platform); proprietary: AWS IoT, Azure IoT, MindSphere, ThingWorx, Ignition (Inductive Automation).

## 🧮 Solved examples

### Example 1: Mosquitto test.

```
# terminal 1: subscribe to every topic under plant1
mosquitto_sub -h broker.example.com -p 8883 --cafile ca.crt \
  -u op1 -P secret -t "plant1/#" -v

# terminal 2: publish a reading
mosquitto_pub -h broker.example.com -p 8883 --cafile ca.crt \
  -u press01 -P secret -t "plant1/line1/press/temp" -m '{"temp":62.4}' -q 1
```

### Example 2: Data model for a CNC machine (JSON).

```
{
  "id": "plant1.line1.cnc03",
  "attributes": {"make": "XYZ", "model": "VMC-850", "installed": "2023-04-10"},
  "telemetry": {"spindle_rpm": 8000, "spindle_load_pct": 46.5,
                "vibration_mm_s": 2.1, "ts": "2026-10-26T09:30:00Z"},
  "state": "RUNNING",
  "commands": ["start", "stop", "set_feed_override"]
}
```

**Example 3: Edge filtering saving.** A vibration sensor samples at 1 kHz with 4 bytes per sample = 4 KB/s = about 346 MB per day. If the gateway sends only an RMS value and peak every second (8 bytes/s), the upload is about 0.69 MB per day, a reduction of about 500 times.

## Key points to remember

- Platform functions: connect, manage, ingest, store, rules, analytics, visualize, integrate, secure.
- Data model: telemetry, attributes, events, commands; AAS, OPC UA, Sparkplug B, WoT.

- Thing abstraction: properties, events, actions.
- Gateway: protocol conversion, edge processing, buffering, security.
- Mosquitto: open-source broker; ports 1883 and 8883; ACLs; bridging.

## ⭐ Important questions

1. What is an IoT platform? Explain its basic functionalities with examples.
2. Explain data modelling and the thing abstraction in IoT.
3. Explain IoT networks used in industry.
4. Explain IoT gateways and machine interfaces.
5. Explain cloud-based Mosquitto brokers and programming IoT applications with free and proprietary software.

NEXT ENGINEER
_sheerat_

# UNIT 5 Machine Learning for Smart Manufacturing

## ✍️ 2-mark definitions

- **Machine learning (ML):** algorithms that learn patterns from data to make predictions or decisions without being explicitly programmed for each case.
- **Supervised learning:** learning from labelled data (input with known output): classification and regression.
- **Unsupervised learning:** finding structure in unlabelled data: clustering, dimensionality reduction, anomaly detection.
- **Self-learning (reinforcement / self-supervised):** the system learns from its own interactions or generates its own labels: reinforcement learning learns by reward; self-supervised learning predicts parts of the data from other parts.
- **Feature learning (representation learning):** automatically learning useful features from raw data, as deep neural networks do.
- **Artificial neural network (ANN):** a model of layers of connected neurons, each computing a weighted sum followed by an activation function, trained by backpropagation.
- **Decision tree:** a tree model that splits data on feature values to predict a class or value.
- **Regression analysis:** modelling the relationship between a dependent variable and one or more independent variables (linear regression $y = a + bx$).
- **Genetic algorithm (GA):** an optimization method inspired by evolution, using selection, crossover and mutation on a population of solutions.
- **Predictive maintenance:** using data and ML to predict equipment failure before it happens, so maintenance is done just in time.

## 💡 Core concepts

### Applications of ML in manufacturing

| Application | ML type | Example |
|---|---|---|
| Predictive maintenance | Supervised regression (remaining useful life), classification (fail / no fail) | Vibration and temperature of bearings |
| Anomaly detection | Unsupervised (clustering, autoencoders) | Unusual motor current patterns |
| Visual quality inspection | Supervised deep learning (CNN) | Detecting scratches on parts |
| Process optimization | Regression, GA, reinforcement learning | Best temperature and pressure for yield |
| Demand forecasting | Time-series regression | Production planning |
| Energy optimization | Regression, RL | HVAC and compressor scheduling |

| Application | ML type | Example |
|---|---|---|
| Robot control | Reinforcement learning | Pick-and-place learning |

**ML workflow:** define problem → collect data (sensors, MES) → clean and label → feature engineering → split train/validation/test → train model → evaluate (accuracy, precision, recall, F1, RMSE) → deploy (edge or cloud) → monitor and retrain (model drift).

**ANN:** input layer (features), hidden layers (ReLU, sigmoid), output layer (softmax for classes, linear for regression); trained by gradient descent and backpropagation; deep networks (CNN for images, RNN/LSTM for time series). Needs a lot of data, less interpretable.

**Decision trees:** choose splits that maximize information gain (entropy) or reduce Gini impurity; easy to interpret; prone to overfitting (use pruning or random forests and gradient boosting).

**Regression:** linear $y = a + bx$ (least squares: $b = \Sigma(x - \bar{x})(y - \bar{y})/\Sigma(x - \bar{x})^2$, $a = \bar{y} - b\,\bar{x}$); multiple regression; logistic regression for classification; evaluation by $R^2$ and RMSE.

**Genetic algorithms:** encode solutions as chromosomes, evaluate fitness, select parents (roulette, tournament), crossover, mutate, repeat; used for scheduling (job shop), layout and parameter optimization.

**Programming tools:** free: Python (scikit-learn, TensorFlow, PyTorch, pandas), KNIME, Orange, Weka; proprietary: MATLAB, AWS SageMaker, Azure ML, Siemens Industrial Edge AI apps.

## 🧮 Solved examples

**Example 1: Linear regression.** Spindle temperature x (relative units) and tool wear y:

| x | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| y | 2 | 4 | 5 | 4 | 5 |

- $\bar{x} = 3$, $\bar{y} = 4$.
- $\Sigma(x - \bar{x})(y - \bar{y}) = (-2)(-2) + (-1)(0) + 0 + (1)(0) + (2)(1) = 6$; $\Sigma(x - \bar{x})^2 = 10$.
- $b = 0.6$, $a = 4 - 0.6 \times 3 = 2.2$. **$\hat{y} = 2.2 + 0.6x$**; prediction at $x = 6$: 5.8.

**Example 2: Entropy for a decision tree split.** 10 parts: 6 good, 4 defective. Entropy = $-0.6 \log_2 0.6 - 0.4 \log_2 0.4 = 0.971$. Split on "temperature high?": high → 1 good, 3 defective (entropy 0.811); normal → 5 good, 1 defective (entropy 0.650). Weighted = $0.4 \times 0.811 + 0.6 \times 0.650 = 0.714$. **Information gain = 0.971 − 0.714 = 0.257**.

**Example 3: Classification metrics.** A defect detector on 100 parts: TP = 18, FP = 2, FN = 2, TN = 78. Precision = 18/20 = 0.90; recall = 18/20 = 0.90; accuracy = 96/100 = 0.96.

## Code

**Predictive maintenance classifier (Python, scikit-learn)**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

df = pd.read_csv("machine_data.csv")          # vibration, temp, current, failed
X = df[["vibration", "temp", "current"]]
y = df["failed"]

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=1)
model = DecisionTreeClassifier(max_depth=4).fit(X_tr, y_tr)
print(classification_report(y_te, model.predict(X_te)))
```

### Key points to remember

- Supervised (labelled), unsupervised (clusters, anomalies), self-learning (RL, self-supervised), feature learning (deep nets).
- Models: ANN, decision trees, regression, GA.
- Workflow: data → features → train → evaluate → deploy → monitor.
- Least squares b = Sxy/Sxx; information gain; precision and recall.

### ⭐ Important questions

1. Explain the foundations of machine learning: supervised, unsupervised, self-learning and feature learning.
2. Explain ANN, decision trees, regression analysis and genetic algorithms with manufacturing examples.
3. Explain applications of ML in smart manufacturing (predictive maintenance, quality inspection).
4. Explain programming ML for manufacturing with free and proprietary software.

## Last-hour checklist

- ☐ Industry 1.0–4.0 (and 5.0); nine pillars; design principles; globalization issues.
- ☐ Lean principles, TIMWOOD, 5S, poka-yoke; OEE (71.3%); takt time (2 min).
- ☐ Smart and connected business; smart factory characteristics.
- ☐ PLC architecture, scan cycle, IEC 61131-3; start-stop ladder; timers; analog scaling (6 bar).
- ☐ Smart sensors, IO-Link; HMI principles; CPS components; ISA-95 table; 5C table; digital twin.
- ☐ MQTT history, packets, QoS table, topics and wildcards, retained, LWT.
- ☐ OPC UA history, address space, client-server, security, tools; protocol comparison table; AR uses.
- ☐ IoT platform functions; data modelling; thing abstraction; gateways; Mosquitto commands; edge saving.
- ☐ ML types; manufacturing applications; ANN, trees, regression ($\hat{y}$ = 2.2 + 0.6x), GA; info gain 0.257; precision and recall.

## Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

| # | Title | Author(s) | Publisher | Where to get it legally |
|---|---|---|---|---|
| 1 | The Concept Industry 4.0: An Empirical Analysis of Technologies and Applications in Production Logistics | Christoph Jan Bartodziej | Springer Gabler (2017) | WorldCat |
| 2 | Industry 4.0: The Industrial Internet of Things | Alasdair Gilchrist | Apress / Springer (2016) | WorldCat |
| 3 | Building the Internet of Things with IPv6 and MIPv6 | Daniel Minoli | Wiley | WorldCat |
| 4 | Architecting the Internet of Things | Dieter Uckelmann, Mark Harrison, Florian Michahelles (eds.) | Springer | WorldCat |
| 5 | The Internet of Things: Connecting Objects to the Web | Hakima Chaouchi | Wiley | WorldCat |
| 6 | The Internet of Things: Key Applications and Protocols (2nd ed.) | Olivier Hersent, David Boswarthick, Omar Elloumi | Wiley | WorldCat |
| 7 | Programmable Logic Controllers (4th ed.) | W. Bolton | Elsevier / Newnes (2006) | WorldCat |

**Syllabus source:** AKTU Open Elective syllabus for B.Tech final year, 2025-26 (copy hosted by an affiliated college, PDF)